

Consistent Hashing With Memcached

Consistent Hashing is a method that's widely used to reduce cache invalidation. Let's take a closer look at how it can be used.



Memcached is a popular in-memory, distributed key-value store that is frequently used as a caching layer (especially for websites). It was developed in 2003 by Brad Fitzpatrick for hosting his website LiveJournal. Since then it has become extremely popular and is being used in Facebook, Zynga and Wikipedia.

Distributing keys and values

Memcached is a distributed key-value store, which means that it distributes the key-value pairs across multiple cache instances.

Consistent hashing is a method of distributing data across multiple cache instances such that an addition or a removal of a node causes less disruption in the cache hits.

The way Memcached distributes the key-values is pretty simple, if there are multiple Memcached instances:

1. For a given key, the client creates a hash ($\text{hash}(\text{key})$) and then maps it to a particular hash instance using the

modulo operation - $\text{hash}(\text{key}) \% \text{number of instances}$.

2. The client stores the value in the instance that matches the result of the above operation.

Simple enough, right? But let us say that we have reached a stage where the existing instances of the cache have outgrown the amount of data they can cache – for instance, if your subscribers have grown 10-fold and the number of hits has gone up 20-fold. The logical thing to do would be to increase the number of cache instances. And therein lies the problem—every time a new instance is introduced, the second variable in the above operation (the number of instances) changes. And when that happens, a key previously mapped to one instance would now be mapped to another.

Let me illustrate that. Let's assume there are 10 instances of Memcached. Let me try to store a key/value into this cluster. Let me also assume that the key ('Hello') produces a hash of 12356 (hashes are much longer—large enough to